

Python for AI & Machine Learning

Class 14 – Inheritance in Python

Trainer: Surya Dekshith Mallikarjuna

Course: Python for AI & Machine Learning

Duration: 2 Hours (Theory + Practical)

Learning Objectives

After completing this session, students will be able to:

- Understand the concept of Inheritance.
 - Explain the relationship between Parent and Child classes.
 - Create classes using inheritance.
 - Use the `super()` function.
 - Understand the different types of inheritance supported in Python.
 - Apply inheritance in AI & Machine Learning and real-world applications.
-

Revision of Previous Class

In the previous class, we learned:

- Encapsulation
- Data Hiding
- Public Members
- Protected Members
- Private Members
- Getter Methods
- Setter Methods

Today, we will learn the **second pillar of Object-Oriented Programming (OOP): Inheritance**.

What is Inheritance?

Inheritance is an Object-Oriented Programming concept where one class acquires the properties (variables) and behaviors (methods) of another class.

The existing class is called the **Parent Class (Base Class)**.

The new class is called the **Child Class (Derived Class)**.

Inheritance allows us to reuse existing code instead of writing it again.

Why Do We Need Inheritance?

Without inheritance:

- The same code is repeated in multiple classes.
- Programs become longer.
- Maintenance becomes difficult.
- More development time is required.

With inheritance:

- Code is reused.
 - Programs become shorter and cleaner.
 - Maintenance becomes easier.
 - Development becomes faster.
 - Applications become easier to extend.
-

Real-Life Example

Consider a **Vehicle**.

Every vehicle has common properties such as:

- Brand
- Color
- Speed

Every vehicle also has common behaviors such as:

- Start
- Stop

Different vehicles like:

- Car
- Bike
- Bus

share these common features but also have their own specific features.

Instead of writing the same code repeatedly, we create one `Vehicle` class and allow other classes to inherit from it.

Parent Class and Child Class

A **Parent Class** contains common functionality.

A **Child Class** inherits all accessible members of the parent class and can also add its own features.

Basic Syntax

```
```python id="y4tqwr" class Parent: pass
```

```
class Child(Parent): pass
```

The child class automatically inherits the public and protected members of the parent class.

---

# Example 1 - Basic Inheritance

```
```python id="q4z8yv"
```

```
class Animal:
```

```
    def eat(self):
```

```
        print("Animal is eating")
```

```
class Dog(Animal):
```

```
    def bark(self):
```

```
        print("Dog is barking")
```

```
dog = Dog()
```

```
dog.eat()
```

```
dog.bark()
```

Output

```
``` id="q98zva" Animal is eating Dog is barking
```

The `Dog` class inherits the `eat()` method from the `Animal` class.

---

# Constructor Inheritance

Child classes also inherit constructors from parent classes.

### Example

```
```python id="qbyqvz"
class Person:

    def __init__(self, name):
        self.name = name

class Student(Person):

    def display(self):
        print("Student Name:", self.name)

student = Student("Rahul")

student.display()
```

Output

```
``` id="p33s8g" Student Name: Rahul
```

---

# The super() Function

The `super()` function is used to call the constructor or methods of the parent class.

It helps avoid rewriting initialization code.

### Example

```
```python id="7gqvmr"
class Person:

    def __init__(self, name):
        self.name = name

class Student(Person):

    def __init__(self, name, course):
        super().__init__(name)
        self.course = course

    def display(self):
        print("Name:", self.name)
        print("Course:", self.course)
```

```
student = Student("Rahul", "AI & ML")

student.display()
```

Output

```
``` id="iv7qv0" Name: Rahul Course: AI & ML
```

```

Types of Inheritance

Python supports the following inheritance types:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

1. Single Inheritance

One child class inherits from one parent class.

Structure

```text
Animal
 |
Dog
```

Example

```
```python id="txuyha" class Animal:
```

```
def sound(self):
 print("Animal makes a sound")
```

```
class Dog(Animal):
```

```
def bark(self):
 print("Dog barks")
```

```
dog = Dog()
```

```
dog.sound() dog.bark()
```

```

2. Multiple Inheritance

One child class inherits from two or more parent classes.

Structure

```text  
Teacher      Researcher  
  \          /  
   Professor
```

Example

```
```python id="ysj6c0" class Teacher:
```

```
def teach(self):
 print("Teaching")
```

```
class Researcher:
```

```
def research(self):
 print("Research")
```

```
class Professor(Teacher, Researcher): pass
```

```
prof = Professor()
```

```
prof.teach() prof.research()
```

```

3. Multilevel Inheritance

A child class becomes the parent of another child class.

Structure

```text  
Animal  
 |  
Mammal
```

```
|  
Dog
```

Example

```
```python id="xg83zy" class Animal:
```

```
def eat(self):
 print("Eating")
```

```
class Mammal(Animal):
```

```
def walk(self):
 print("Walking")
```

```
class Dog(Mammal):
```

```
def bark(self):
 print("Barking")
```

```
dog = Dog()
```

```
dog.eat() dog.walk() dog.bark()
```

```

4. Hierarchical Inheritance

Multiple child classes inherit from one parent class.

Structure

```text  
    Vehicle  
    /  
   Car    Bike
```

Example

```
```python id="rvrvvj" class Vehicle:
```

```
def start(self):
 print("Vehicle Started")
```

```
class Car(Vehicle):
```

```
 def drive(self):
 print("Car Driving")
```

```
class Bike(Vehicle):
```

```
 def ride(self):
 print("Bike Riding")
```

```
car = Car() bike = Bike()
```

```
car.start() car.drive()
```

```
bike.start() bike.ride()
```

```

```

```
5. Hybrid Inheritance
```

```
Hybrid inheritance combines two or more inheritance types.
```

```
Structure
```

```
```text
```

```
    A  
    /  
B   C  
 \  /  
  D
```

Python supports hybrid inheritance using multiple inheritance.

AI Example – Machine Learning Model

```
```python id="55ukpo" class MLModel:
```

```
 def load_data(self):
 print("Loading Dataset")
```

```
class ClassificationModel(MLModel):
```

```
def train(self):
 print("Training Classification Model")
```

```
model = ClassificationModel()
```

```
model.load_data() model.train()
```

```
Output

```text  
Loading Dataset  
Training Classification Model
```

AI Example – Deep Learning

```
```python id="w59l6g" class NeuralNetwork:
```

```
def initialize(self):
 print("Initializing Neural Network")
```

```
class CNN(NeuralNetwork):
```

```
def train(self):
 print("Training CNN Model")
```

```
cnn = CNN()
```

```
cnn.initialize() cnn.train()
```

```
Output

```text  
Initializing Neural Network  
Training CNN Model
```

Real-World Example – Employee Management System

```
```python id="kgqunm" class Employee:
```

```
def company(self):
 print("Company: Digix Technologies")
```

```
class Developer(Employee):
```

```
 def role(self):
 print("Role: AI Developer")
```

```
developer = Developer()
```

```
developer.company() developer.role()
```

```
Output
```

```
```text  
Company: Digix Technologies  
Role: AI Developer
```

Advantages of Inheritance

- Promotes code reuse.
- Reduces duplicate code.
- Simplifies maintenance.
- Improves code organization.
- Supports extensibility.
- Speeds up software development.
- Makes large applications easier to manage.

Applications of Inheritance

Inheritance is commonly used in:

- AI & Machine Learning frameworks
 - Deep Learning models
 - Web development
 - Game development
 - Banking systems
 - Hospital management systems
 - School management software
 - Enterprise Resource Planning (ERP)
 - Customer Relationship Management (CRM)
-

Summary

In today's session, we learned:

- Introduction to Inheritance
- Parent Class and Child Class
- Code Reusability
- Constructor Inheritance
- The `super()` Function
- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance
- AI and real-world examples

Inheritance is a fundamental concept in Object-Oriented Programming that helps developers build scalable, reusable, and maintainable applications.

Practical Exercises

Practical 1 – Vehicle System

Create a `Vehicle` class with:

Methods:

- Start
- Stop

Create a `Car` class that inherits from `Vehicle`.

Add a method:

- Drive
-

Practical 2 – Employee System

Create an `Employee` class.

Methods:

- Login
- Logout

Create a `Manager` class.

Method:

- Approve Leave
-

Practical 3 – AI Model

Create an `MLModel` class.

Method:

- Load Dataset

Create a `RegressionModel` class.

Methods:

- Train
 - Predict
-

Practical 4 – Banking System

Create a `BankAccount` class.

Methods:

- Deposit
- Withdraw

Create a `SavingsAccount` class.

Method:

- Calculate Interest
-

Assignment

Develop a **University Management System** using Inheritance.

Parent Class

`Person`

Attributes:

- Name

- Age

Method:

- Display Details

Child Class 1

Student

Additional Attributes:

- Roll Number
- Course

Child Class 2

Teacher

Additional Attributes:

- Employee ID
- Subject

Requirements:

- Use the `super()` function to initialize common attributes.
- Create at least three student objects and two teacher objects.
- Display all details using inherited methods.

Interview Questions

1. What is Inheritance?
2. What is the difference between a Parent Class and a Child Class?
3. What are the advantages of Inheritance?
4. What is the purpose of the `super()` function?
5. What is Single Inheritance?
6. What is Multiple Inheritance?
7. What is Multilevel Inheritance?
8. What is Hierarchical Inheritance?
9. What is Hybrid Inheritance?
10. How is Inheritance used in AI and Machine Learning libraries?

Next Class Preview

In the next session, we will learn **Polymorphism**, the third pillar of Object-Oriented Programming.

Topics include:

- What is Polymorphism?
- Method Overriding
- Method Overloading (Python approaches)
- Runtime Polymorphism
- Duck Typing
- Operator Overloading
- AI & Machine Learning examples
- Real-world implementation